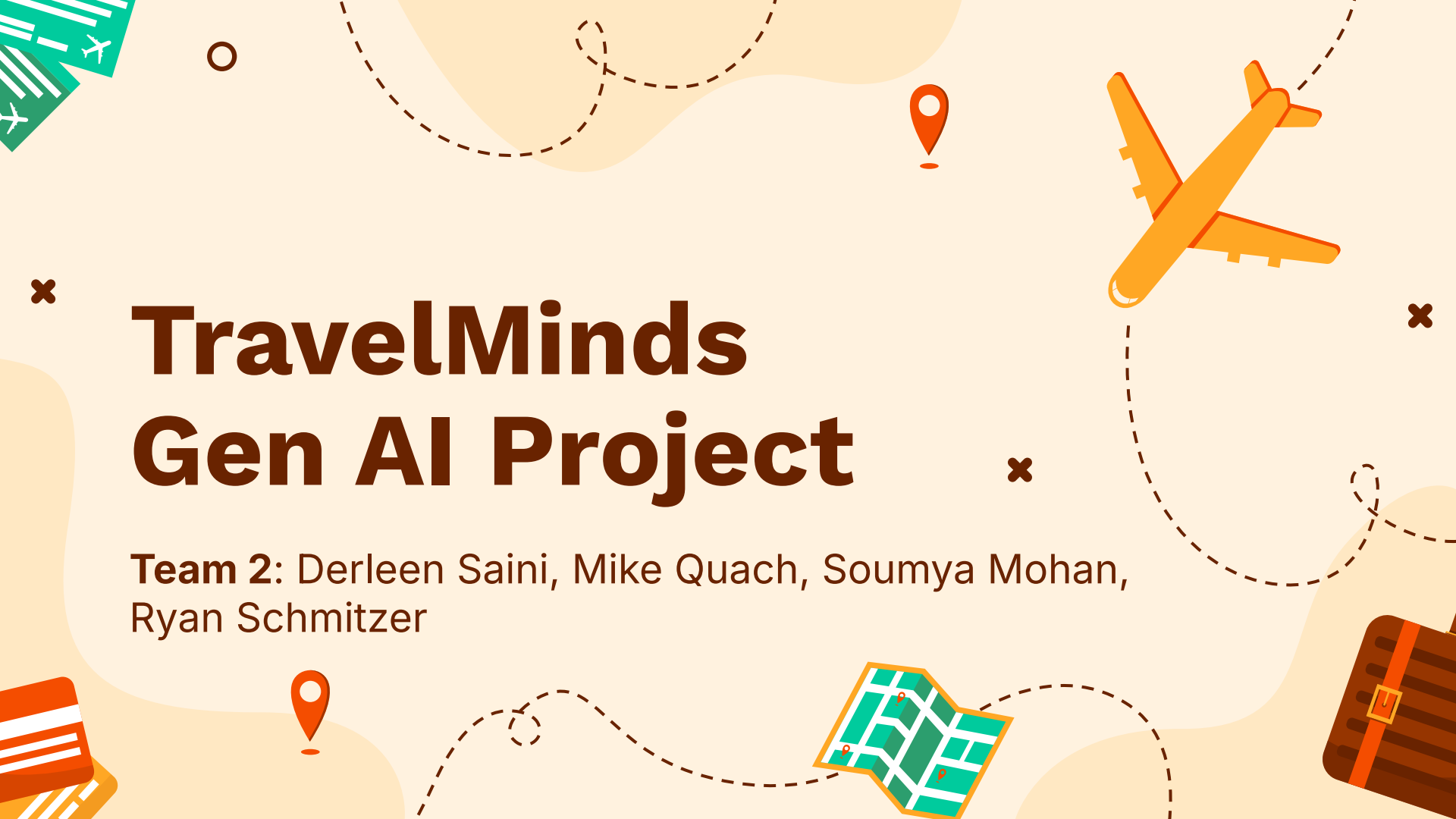




TravelMinds Gen AI Project

Team 2: Derleen Saini, Mike Quach, Soumya Mohan,
Ryan Schmitzer



Table of contents

01

Introduction

What is TravelMinds?

02

Business Problem and Solution

What problem are we
solving

03

Architecture

Gen AI model

04

Data/ Product Demo

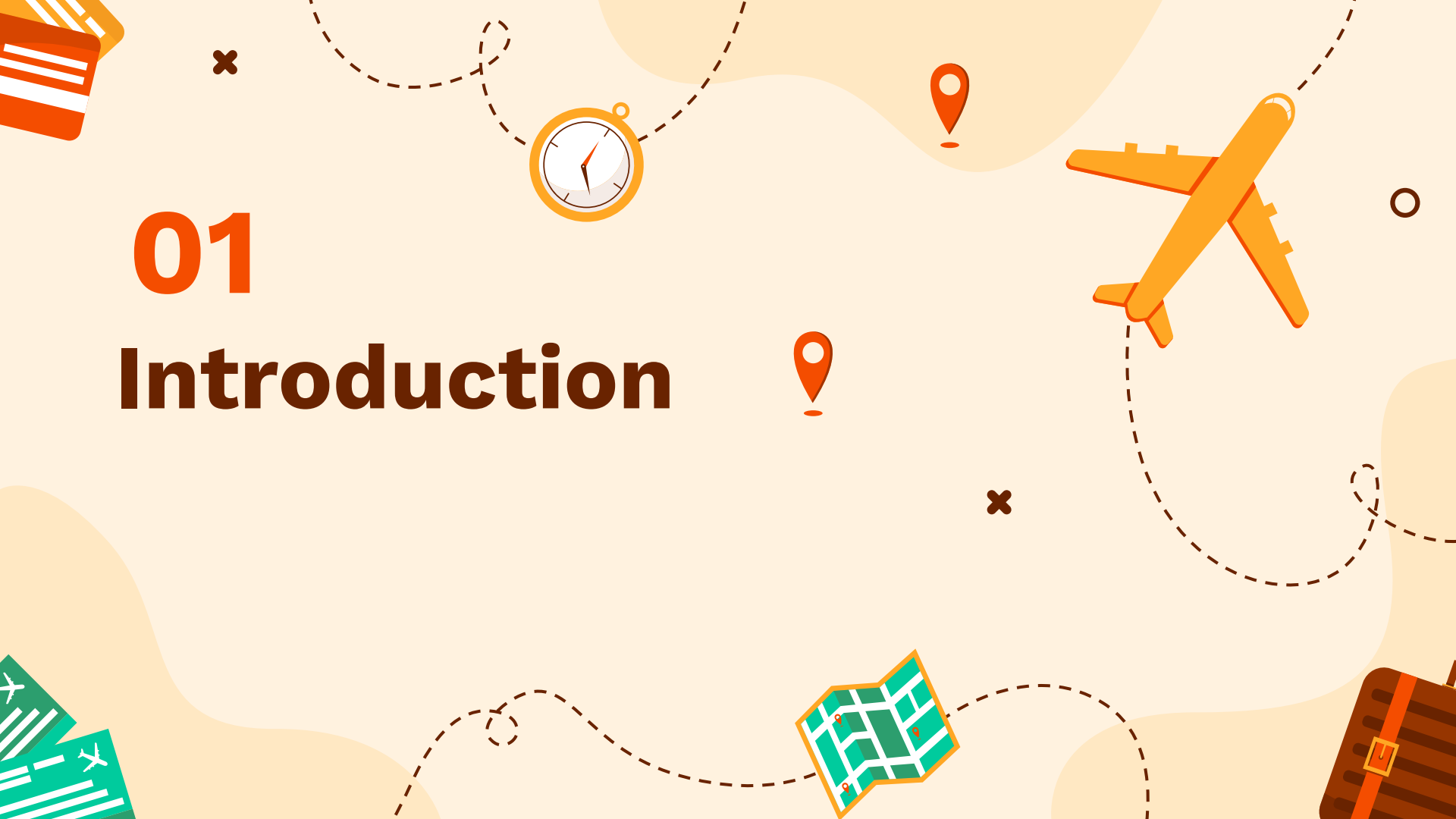
05

Improvements

Overview and future
potential

06

Conclusion and Q&A



01

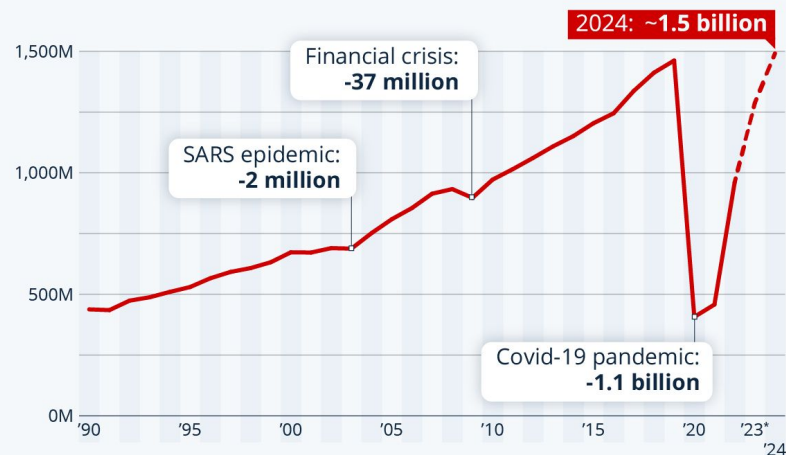
Introduction

Introduction

- 9.6% of Global GDP
 - Global Travel ~2 trillion market
- 52% of Americans are putting aside \$4,000+
- 5.8% annual growth

International Tourism to Surpass Pre-Pandemic Levels in 2024

International tourist arrivals worldwide since 1990



* 2023 figure is provisional, 2024 figure as forecast in Jan. 2024

Source: UNWTO



statista

02 **Business** **Problem and** **Proposed** **Solution**



Business Problem

Issue:

- Inefficient travel planning
 - Time Consuming
 - 20 - 30 hrs
 - Cost
- Planning itineraries around the shortlisted attractions



Business Problems

Solution

Opportunity

- Leverage Generative AI
 - Increase efficiency
 - Customization

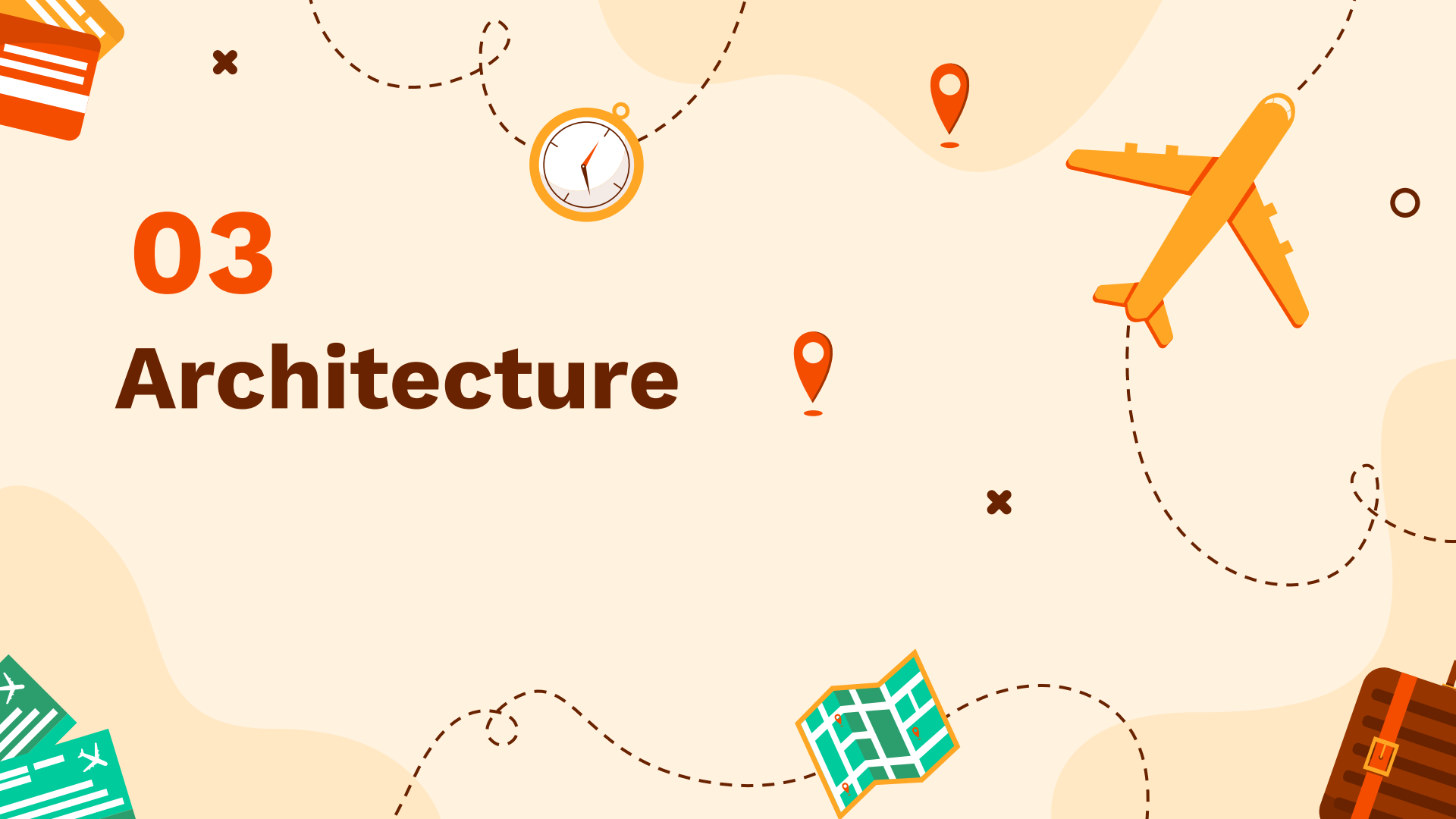
Proposed Solution

- Match user interests with relevant attractions
- Improves the travel experience

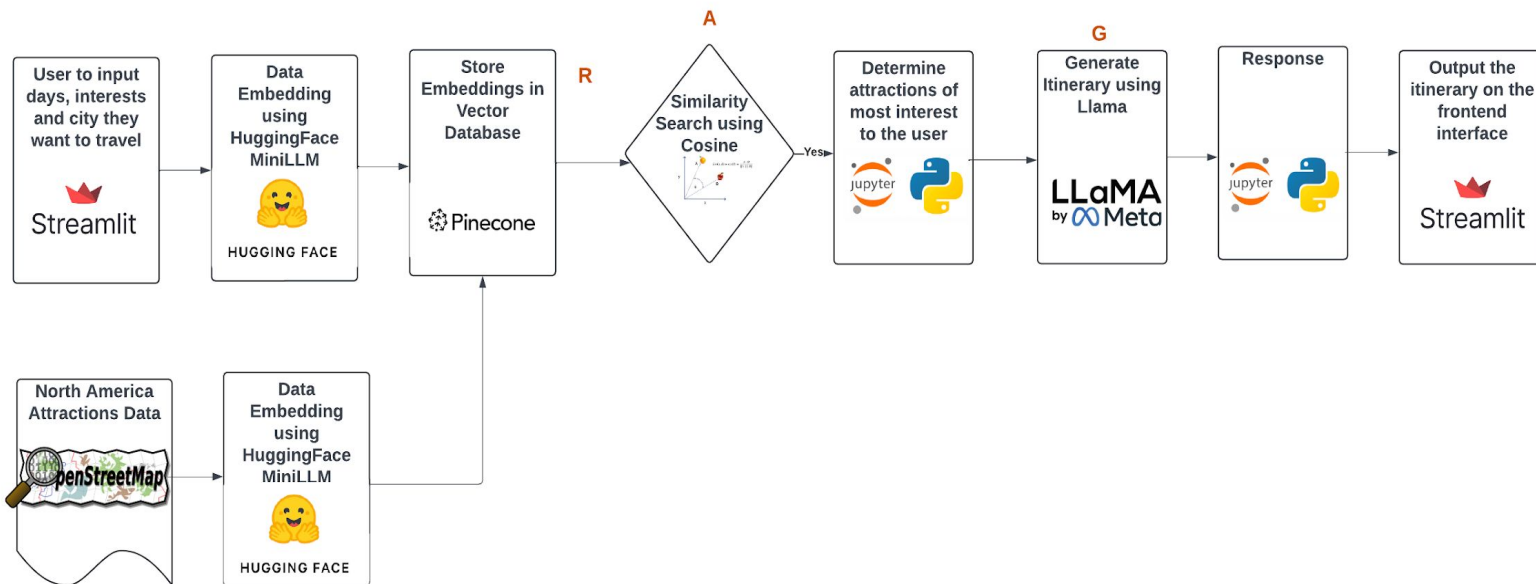


03

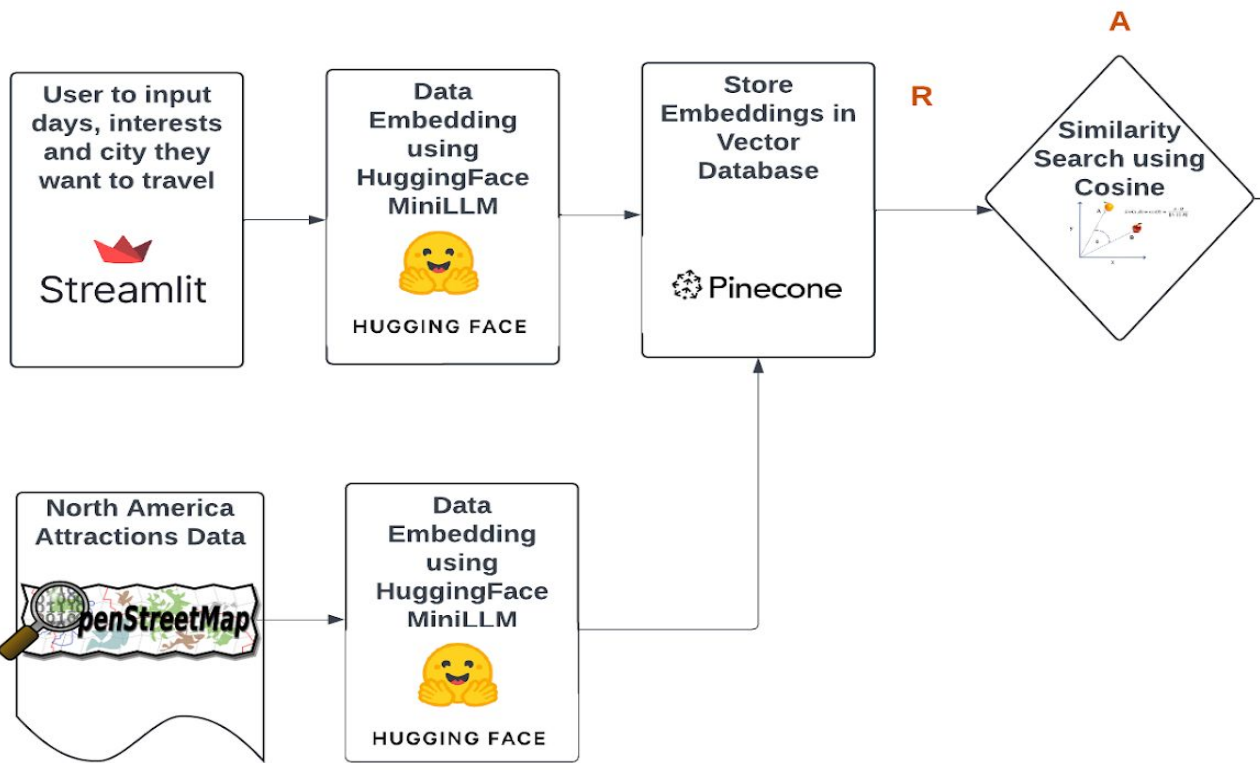
Architecture



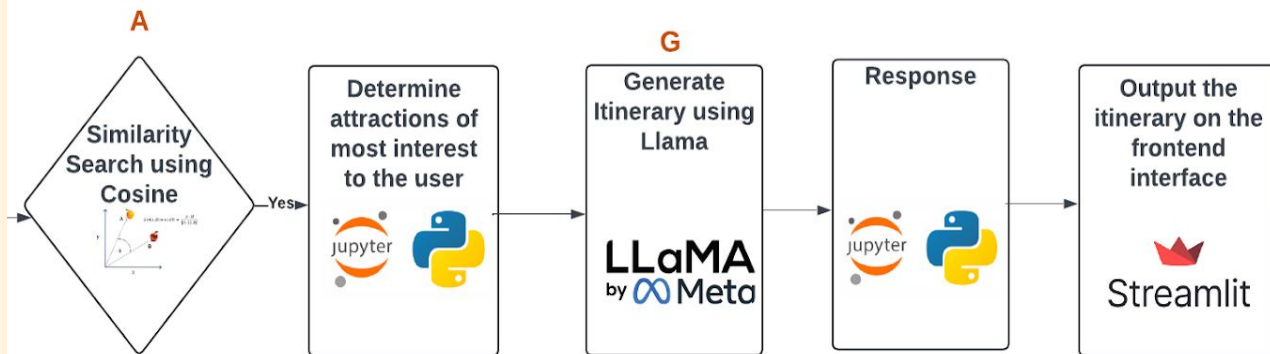
Architecture



Architecture



Architecture





04

Backend & Data



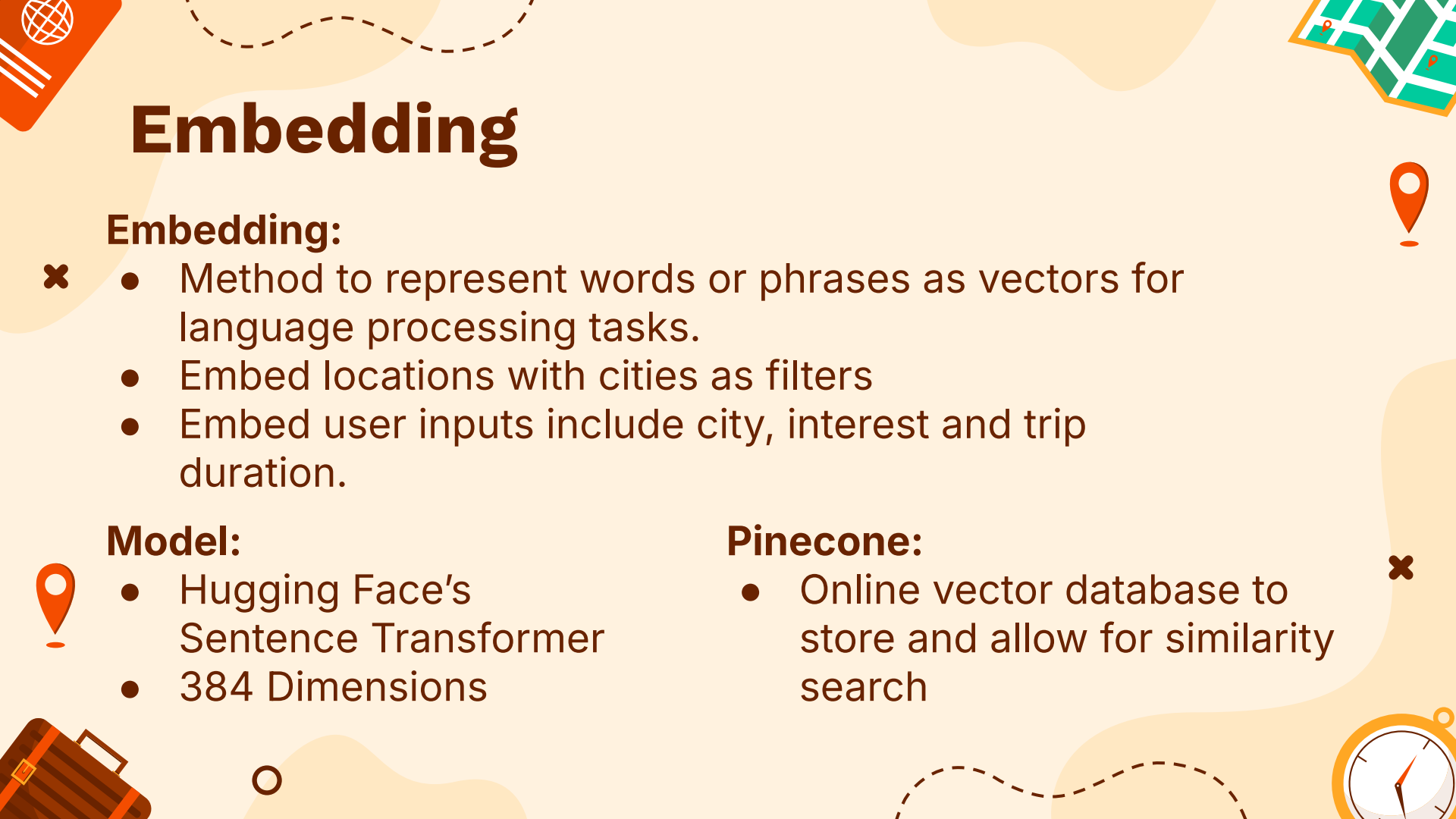
Data

Source: Open Street Map

59
Attraction
Type

6808
Locations

3286
cities



Embedding

Embedding:

- ✕ • Method to represent words or phrases as vectors for language processing tasks.
- Embed locations with cities as filters
- Embed user inputs include city, interest and trip duration.



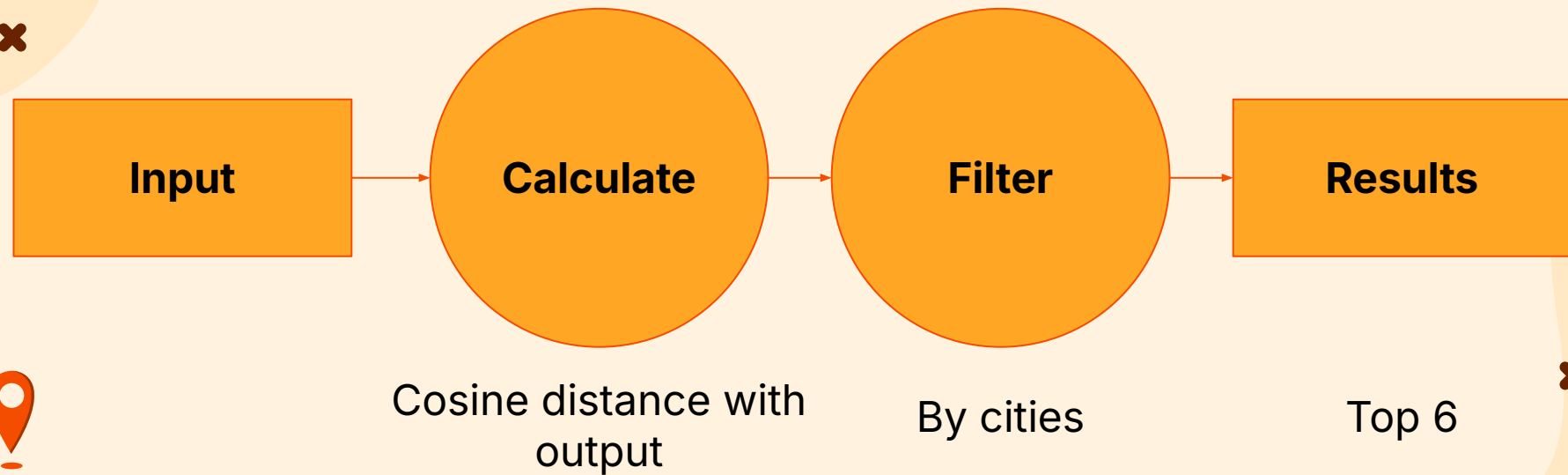
Model:

- Hugging Face's Sentence Transformer
- 384 Dimensions

Pinecone:

- Online vector database to store and allow for similarity search
- 

Semantic Search





04

Product Demo !

Product Demo

```
#!/usr/bin/env python
# coding: utf-8

# In[26]:

import streamlit as st
import os
from dotenv import load_dotenv
env_path = './.env' # Adjust the path if your file is in a different location
load_dotenv(dotenv_path=env_path)
pinecone_key = os.getenv('PINECONE_API_KEY')
from pinecone import Pinecone
from sentence_transformers import SentenceTransformer
import pandas as pd
import ollama

#ollama run llama3

pc = Pinecone(api_key=pinecone_key)
index = pc.Index('mysca')
model = SentenceTransformer('all-MiniLM-L6-v2')

#These are the input the backend requires from the frontend
# change directory for 'csv_file' to where the traveldata.csv is located
csv_file = './traveldata.csv'
num_days = 3
text_prompt = "park"
city = "San Jose"

def query_and_generate_travel_plan(text_prompt, csv_file, city, num_days):
    def generate_travel_plan_with_llama(landmarks, city, num_days):
        # Ensure the number of days does not exceed 3
        num_days = min(num_days, 3)

        # Generate the prompt
        messages = [
            {
                'role': 'user',
                'content': f"""
Generate a travel itinerary with the following details:
- Location: {city}
- Landmarks: {landmarks}
- Number of days: {num_days}

For each day, provide a plan with:
- Morning: Visit one landmark
- Afternoon: Visit another landmark

Use only the provided details. Start the response directly with the itinerary formatted as bullet points for each day without any introductory text or additional information.
"""
            }
        ]

        # Make the request to LLaMA
        response = ollama.chat(model='llama3', messages=messages)
        return response['message'][0]['content']

    # Generate the embedding
    embedding = model.encode(text_prompt)
    vector_as_list = embedding.tolist()

    # Query the Pinecone index
    query_result = index.query(
        namespace='travel_embedding',
        vector=vector_as_list,
```

2024-05-01 14:57:34



05

Improvements !

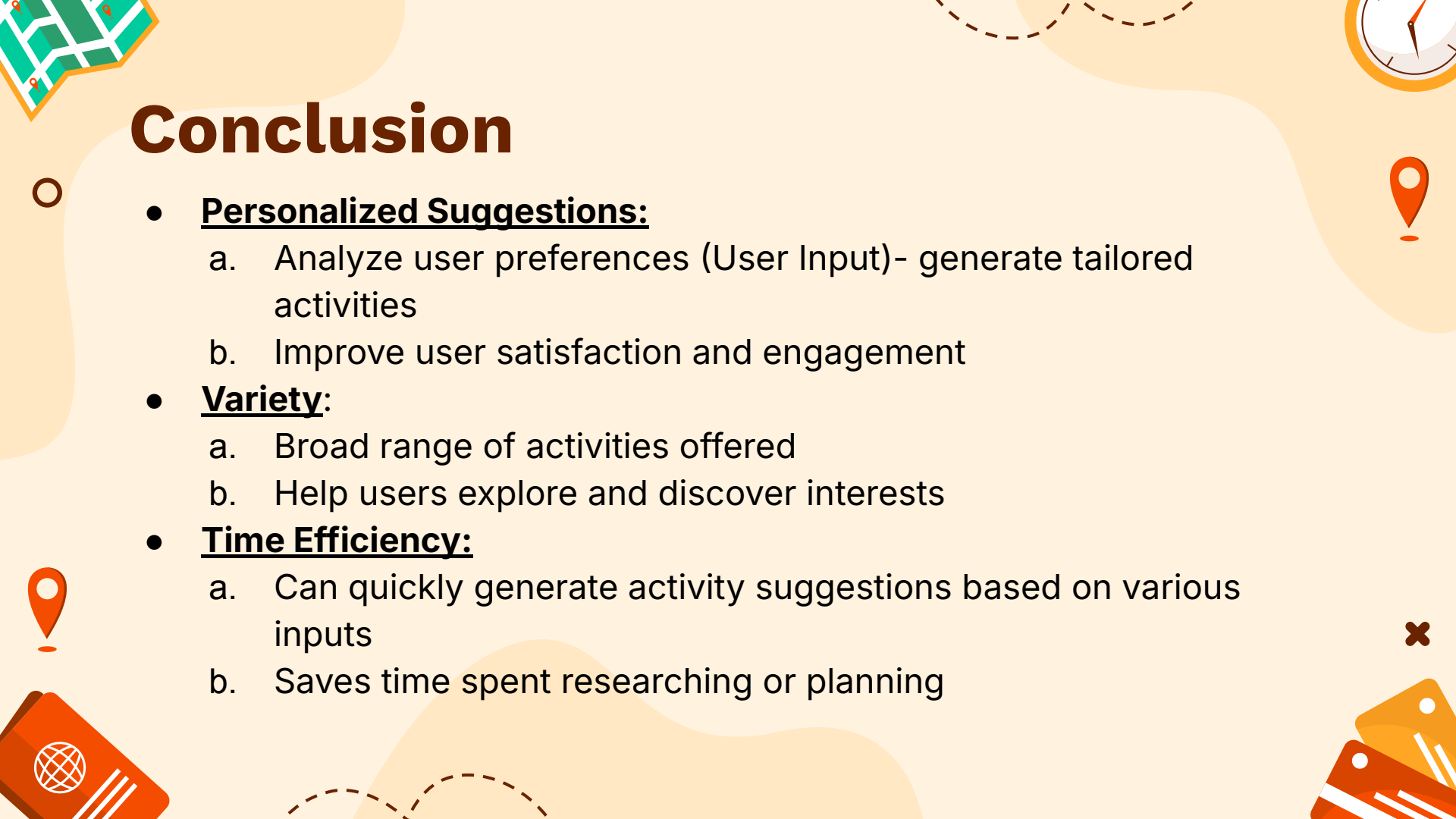
Future Improvements

- **Contextual Recommendations:**
 - a. Time of day, weather, location, and group size, budget
 - b. Enhance suitability and practicality
- **Future Integrations:**
 - a. Weather API
 - b. Google Maps - routes, roads, maps
 - c. Currency Exchange
 - d. Travel API's - Kayak, Expedia, Yelp
- **Adaptive Learning:** The AI can adapt to market trends over time, ensuring its suggestions remain relevant and up-to-date.



06 Conclusion





Conclusion

- **Personalized Suggestions:**
 - a. Analyze user preferences (User Input)- generate tailored activities
 - b. Improve user satisfaction and engagement
- **Variety:**
 - a. Broad range of activities offered
 - b. Help users explore and discover interests
- **Time Efficiency:**
 - a. Can quickly generate activity suggestions based on various inputs
 - b. Saves time spent researching or planning

The background is a light beige color with large, soft, wavy shapes in shades of orange and yellow. Scattered throughout are various travel-related icons: a compass with a white face and orange hands, an orange airplane, a green and white map, a brown suitcase with an orange strap, a green ticket with a white airplane icon, a red location pin, and several small black 'x' marks. Dashed lines suggest a travel path or route.

Thank You!

Any Questions?

Streamlit

```
def main():  
    # Frontend UI  
    image_path = "/Users/derleensaini/Desktop/Travel_Minds_Logo.jpg"  
    col1, col2 = st.columns([1, 5])  
    col1.write("")  
    col2.image(image_path, width = 450)  
    st.markdown(  
        f"<h1 style='text-align: center; font-size: 70px;font-family: URW Chancery L, cursive'>TravelMinds</h1>",  
        unsafe_allow_html=True  
    )  
    st.sidebar.title('User Input')  
    city = st.sidebar.text_input('Enter your destination (City)')  
    num_days = st.sidebar.slider("Duration (days)", min_value=1, max_value=3)  
    text_prompt = st.sidebar.text_area('Enter type of activities you want to do')  
  
    if st.button('Give me Recommendations'):  
        # Call backend functions with user input  
        #resulting_df = query_and_retrieve_data(activities, destination, 'traveldata.csv')  
        #landmarks = "\n".join([f"{row['name']} located in {row['addr_city']}." for _, row in resulting_df.iterrows()])  
        travel_plan = query_and_generate_travel_plan(text_prompt, csv_file, city, num_days)  
  
        # Display results  
        st.header('Suggested Attractions Itinerary')  
        st.write(travel_plan)|  
  
if __name__ == "__main__":  
    main()
```

Product Demo

×

User Input

Enter your destination (City)

Vancouver

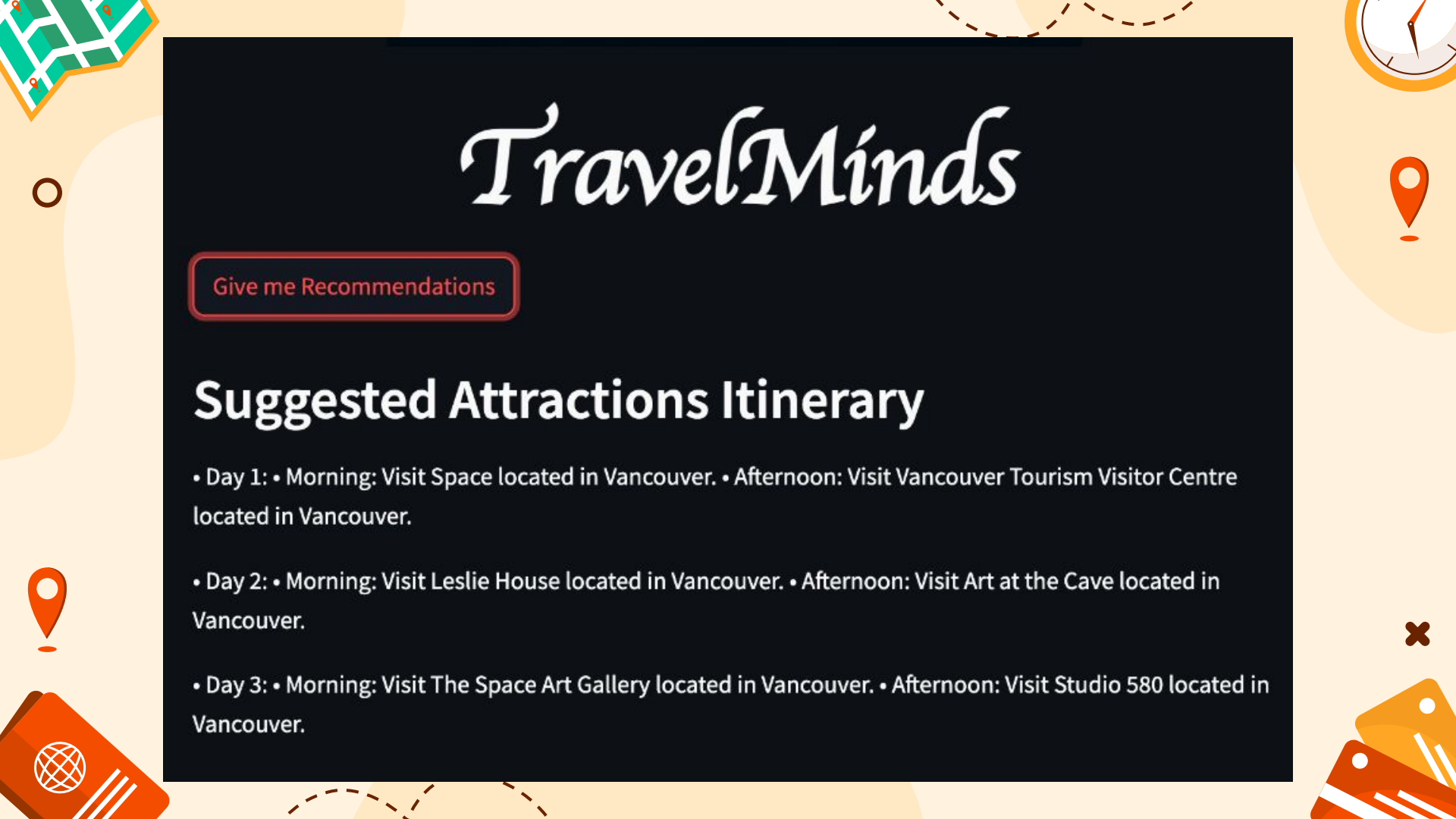
Duration (days)

3

13

Enter type of activities you want to do

history



TravelMinds

[Give me Recommendations](#)

Suggested Attractions Itinerary

- Day 1: • Morning: Visit Space located in Vancouver. • Afternoon: Visit Vancouver Tourism Visitor Centre located in Vancouver.
- Day 2: • Morning: Visit Leslie House located in Vancouver. • Afternoon: Visit Art at the Cave located in Vancouver.
- Day 3: • Morning: Visit The Space Art Gallery located in Vancouver. • Afternoon: Visit Studio 580 located in Vancouver.